

BAB 4: Pemrograman Modular: Prosedur dan Fungsi

1. Pendahuluan

Pada bab-bab sebelumnya, Anda telah mempelajari cara membuat aplikasi yang dapat mengambil keputusan dan melakukan perulangan. Namun, perhatikan kode yang telah Anda tulis sejauh ini. Apakah ada bagian yang terasa **berulang-ulang**? Misalnya, kode untuk mengosongkan TextBox yang Anda tulis di setiap tombol Reset?

Analogi Mesin Pembuat Kopi: Bayangkan Anda memiliki sebuah kios kopi. Setiap kali ada pelanggan yang memesan kopi, Anda harus:

- 1. Mengambil biji kopi
- 2. Menggiling biji kopi
- 3. Menyeduh dengan air panas
- 4. Menuang ke gelas
- 5. Menambahkan gula sesuai permintaan

Jika Anda menulis semua langkah ini setiap kali ada pesanan, Anda akan kelelahan dan rentan membuat kesalahan. Solusinya? Anda membuat **Mesin Pembuat Kopi** yang sudah diprogram dengan semua langkah tersebut. Sekarang, cukup tekan satu tombol "Buat Kopi", mesin akan melakukan semua langkah otomatis.

Dalam pemrograman, **Prosedur dan Fungsi** adalah "mesin" tersebut. Anda menulis kode sekali, lalu memanggilnya berkali-kali dari berbagai tempat dalam aplikasi.

Konsep DRY (Don't Repeat Yourself): Ini adalah prinsip emas dalam pemrograman profesional:

"Every piece of knowledge must have a single, unambiguous, authoritative representation within a system."

Artinya: Setiap logika atau kode hanya boleh ditulis **satu kali** dalam sistem Anda. Mengapa?

Masalah Kode Berulang	Solusi Modular
Sulit maintenance (ubah 10 tempat)	Cukup ubah 1 tempat
Rawan bug (lupa ubah salah satu)	Konsisten di semua tempat
Kode panjang dan membingungkan	Kode pendek dan terorganisir
Sulit dibaca orang lain	Nama prosedur menjelaskan fungsi

Mengapa kita Perlu Memahami Pemrograman Modular? Sebagai calon guru, kita harus membiasakan diri menulis kode yang:

- 1. **Mudah Dipelihara:** Jika ada perubahan kurikulum (misal: KKM berubah), Anda hanya perlu mengubah satu fungsi.
- 2. **Mudah Dibaca Orang Lain:** orang lain yang menggunakan aplikasi Anda dapat memahami logikanya.
- 3. **Efisien:** Aplikasi lebih ringan karena kode yang terorganisir dengan baik.

Prinsip ini mencerminkan sikap **profesionalisme** dan **tanggung jawab** dalam pengembangan perangkat lunak.

2. Capaian Pembelajaran

Kode	Sasaran Pembelajaran (Sub-CPMK)
Sub-CPMK 6	Mahasiswa mampu mengimplementasikan pemrograman modular menggunakan Sub Procedure dan Function, serta mengelola modul untuk meningkatkan efisiensi dan kebersihan kode (Clean Code).

Indikator Pencapaian:

1. Mahasiswa dapat membedakan penggunaan Sub Procedure dan Function dengan tepat.
2. Mahasiswa dapat membuat dan memanggil prosedur/fungsi dengan parameter yang sesuai.
3. Mahasiswa memahami perbedaan ByVal dan ByRef dalam pengiriman parameter.
4. Mahasiswa dapat membuat Module untuk fungsi global yang dapat diakses dari berbagai Form.
5. Mahasiswa menerapkan prinsip Clean Code dalam penamaan dan struktur prosedur/fungsi.

3. Materi 1: Sub Procedure vs Function

3.1. Perbedaan Mendasar

Dalam VB.NET, ada dua jenis modul kode yang dapat Anda buat: **Sub Procedure** dan **Function**. Perbedaan utamanya terletak pada **pengembalian nilai**.

Aspek	Sub Procedure	Function
Kata Kunci	Sub	Function
Mengembalikan Nilai	✗ Tidak	☑ Ya
Penggunaan	Melakukan aksi/tugas	Melakukan kalkulasi & mengembalikan hasil
Pemanggilan	NamaProsedur()	variabel = NamaFungsi()
Contoh	BersihkanForm(), SimpanData()	HitungLuas(), CekLulus()



[GAMBAR 4.1: Diagram Perbedaan Sub vs Function]

3.2. Struktur Penulisan Sub Procedure

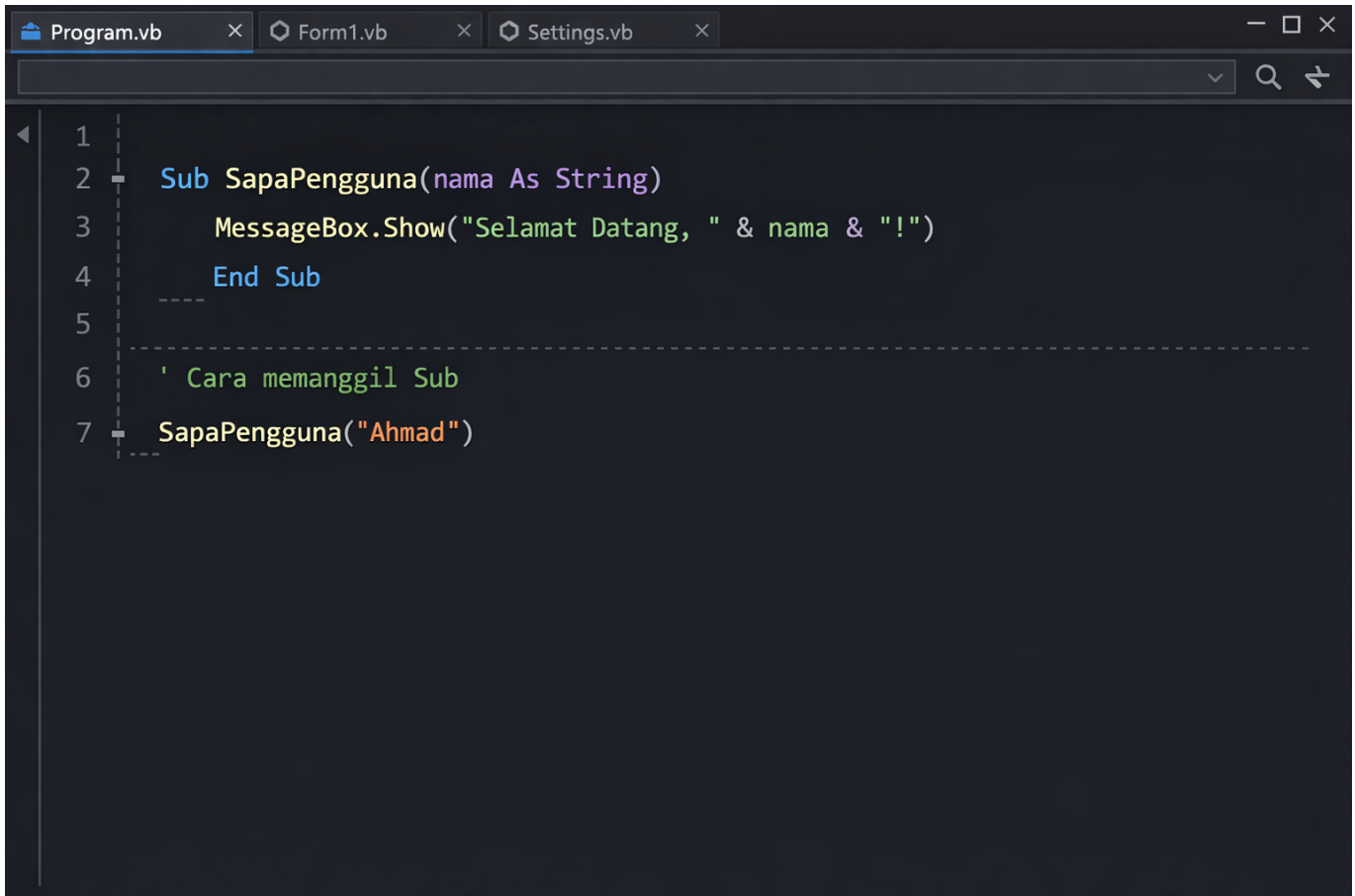
Sintaks Dasar:

```
Sub NamaProsedur([parameter])
    ' Kode yang akan dijalankan
End Sub
```

Contoh: Prosedur untuk Menyapa Pengguna

```
Sub SapaPengguna(nama As String)
    MessageBox.Show("Selamat Datang, " & nama & "!")
End Sub

' Cara memanggil:
SapaPengguna("Ahmad")
```



```
1 |
2 | Sub SapaPengguna(nama As String)
3 |     MessageBox.Show("Selamat Datang, " & nama & "!")
4 | End Sub
5 |
6 | ' Cara memanggil Sub
7 | SapaPengguna("Ahmad")
```

[MASUKKAN GAMBAR 4.2: Contoh Kode Sub Procedure]

3.3. Struktur Penulisan Function

Sintaks Dasar:

```
Function NamaFungsi([parameter]) As TipeDataKembalian
    ' Kode kalkulasi
    Return nilaiHasil
End Function
```

Contoh: Function untuk Menghitung Luas Persegi

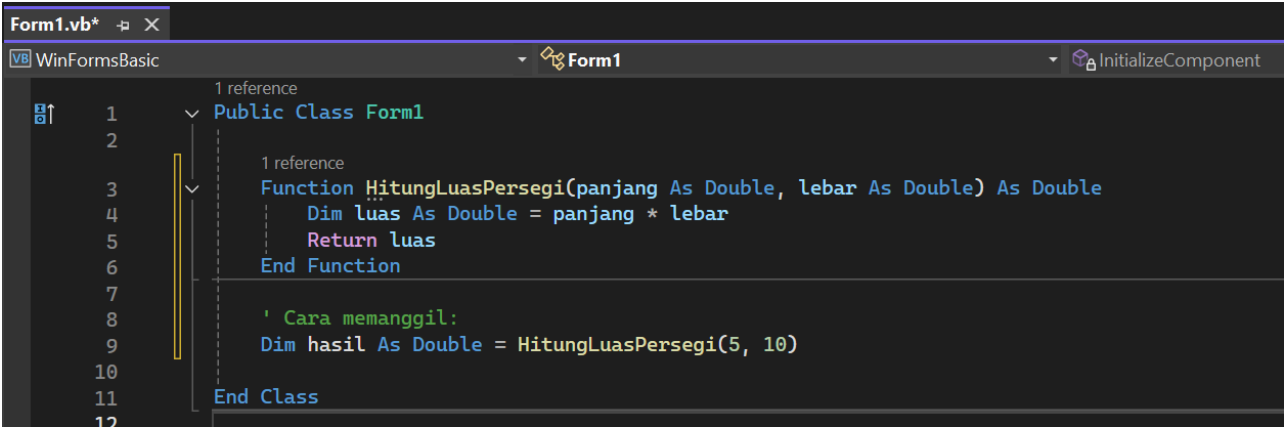
```
Function HitungLuasPersegi(panjang As Double, lebar As Double) As Double
    Dim luas As Double = panjang * lebar
    Return luas
End Function

' Cara memanggil:
Dim hasil As Double = HitungLuasPersegi(5, 10)
```

Kata Kunci Return: Perintah **Return** berfungsi untuk:

1. Mengembalikan nilai kepada pemanggil

2. Mengakhiri eksekusi function



```
1 Public Class Form1
2
3     1 reference
4     Function HitungLuasPersegi(panjang As Double, lebar As Double) As Double
5         Dim luas As Double = panjang * lebar
6         Return luas
7     End Function
8
9     ' Cara memanggil:
10    Dim hasil As Double = HitungLuasPersegi(5, 10)
11
12 End Class
```

[MASUKKAN GAMBAR 4.3: Contoh Kode Function dengan Return] Label: Editor kode yang menunjukkan deklarasi Function dengan Return value.

3.4. Kapan Memilih Sub vs Function?

Gunakan panduan berikut untuk memutuskan:

Situasi	Pilih	Alasan
Menampilkan MessageBox	Sub	Tidak perlu nilai kembali
Mengosongkan TextBox	Sub	Hanya aksi, tidak ada hasil
Menghitung nilai akhir	Function	Perlu hasil perhitungan
Validasi input (True/False)	Function	Perlu status validasi
Menyimpan ke database	Sub	Aksi simpan, tidak perlu hasil
Mengambil data dari database	Function	Perlu mengembalikan data

4. Materi 2: Parameter dan Argumen

4.1. Konsep Parameter vs Argumen

Seringkali kedua istilah ini digunakan bergantian, tetapi ada perbedaan teknis:

Istilah	Definisi	Contoh
Parameter	Variabel yang didefinisikan dalam deklarasi prosedur/fungsi	Sub Hitung(x As Integer)
Argumen	Nilai aktual yang dikirim saat memanggil prosedur/fungsi	Hitung(5)

Parameter vs Argumen



[GAMBAR 4.4: Ilustrasi Parameter vs Argumen]

4.2. Cara Mengirim Data ke Prosedur/Fungsi

Parameter memungkinkan Anda mengirim data ke dalam prosedur/fungsi untuk diproses.

Contoh: Function dengan Multiple Parameter

```
Function HitungRataRata(nilai1 As Double, nilai2 As Double, nilai3 As Double) As Double
    Dim total As Double = nilai1 + nilai2 + nilai3
    Dim rataRata As Double = total / 3
    Return rataRata
End Function

' Memanggil dengan argumen:
Dim hasil As Double = HitungRataRata(80, 85, 90)
' hasil = 85
```

4.3. ByVal vs ByRef

VB.NET menyediakan dua cara untuk mengirim parameter:

Aspek	ByVal (By Value)	ByRef (By Reference)
Arti	Mengirim salinan nilai	Mengirim referensi alamat memori

Aspek	ByVal (By Value)	ByRef (By Reference)
Perubahan di Fungsi	Tidak mempengaruhi variabel asli	Mengubah variabel asli
Default VB.NET	☒ Ya	✗ Harus eksplisit
Keamanan	Lebih aman	Berisiko jika tidak hati-hati

Contoh ByVal (Aman):

```
Sub UbahNilai(ByVal x As Integer)
    x = 100 ' Hanya mengubah salinan
End Sub

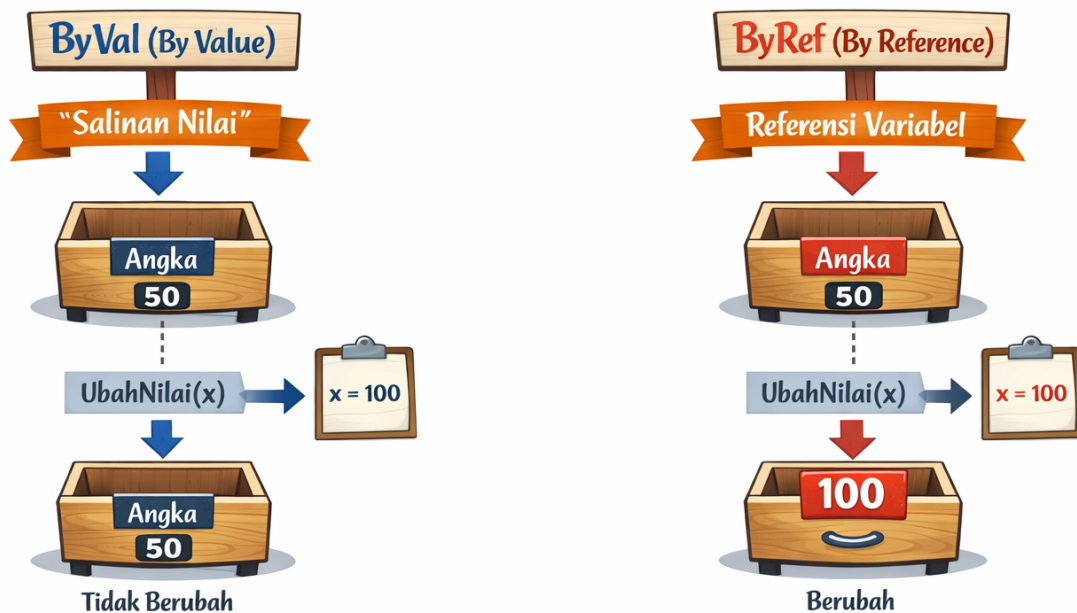
Dim angka As Integer = 50
UbahNilai(angka)
' angka tetap 50
```

Contoh ByRef (Berubah):

```
Sub UbahNilai(ByRef x As Integer)
    x = 100 ' Mengubah variabel asli
End Sub

Dim angka As Integer = 50
UbahNilai(angka)
' angka sekarang 100
```

ByVal vs ByRef



[GAMBAR 4.5: Perbandingan ByVal vs ByRef]

Rekomendasi Best Practice:

- Gunakan **ByVal** sebagai default untuk keamanan data.
- Gunakan **ByRef** hanya ketika Anda **sengaja** ingin mengubah nilai variabel asli (misal: fungsi swap).

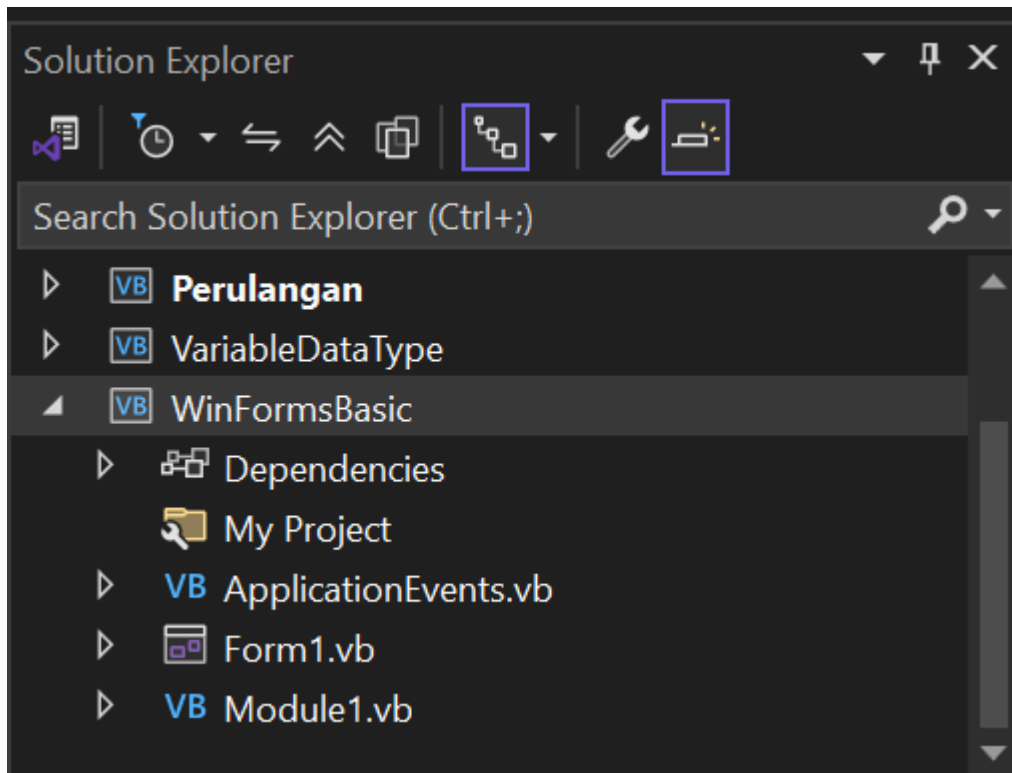
5. Materi 3: Penggunaan Module (.vb)

5.1. Apa itu Module?

Module adalah file khusus (.vb) yang berisi prosedur dan fungsi yang dapat diakses secara **global** dari seluruh proyek Anda. Ini seperti "perpustakaan fungsi" yang bisa dipinjam oleh Form mana saja.

Kapan Menggunakan Module?

- Fungsi yang digunakan di **banyak Form** (misal: fungsi koneksi database)
- Konstanta global (misal: nama sekolah, alamat)
- Fungsi utilitas (misal: format tanggal, validasi email)



[GAMBAR 4.6: Struktur Solution Explorer dengan Module]

5.2. Membuat Module Baru

Langkah-langkah:

1. Klik kanan pada nama Proyek di **Solution Explorer**
2. Pilih **Add** → **Module...**
3. Beri nama, misalnya `modGlobal.vb`
4. Klik **Add**

Struktur Module:

```
Module modGlobal

    ' Konstanta Global
    Public Const NAMA_SEKOLAH As String = "SMAN 1 Pendidikan"

    ' Fungsi Global
    Public Function CekKoneksi() As Boolean
        ' Kode cek koneksi database
        Return True
    End Function

End Module
```

5.3. Memanggil Fungsi dari Module

Keuntungan Module: Anda **tidak perlu membuat instance** untuk memanggil fungsinya.

```

' Dari Form1
Private Sub btnCek_Click(...) Handles btnCek.Click
    ' Langsung panggil tanpa "New"
    If CekKoneksi() = True Then
        MessageBox.Show("Koneksi OK!")
    End If
End Sub

' Dari Form2 (juga bisa!)
Private Sub Form2_Load(...) Handles MyBase.Load
    lblSekolah.Text = NAMA_SEKOLAH
End Sub

```

5.4. Contoh: Fungsi KoneksiCek() di Module

Berikut adalah contoh nyata fungsi yang berguna di aplikasi:

```

Module modDatabase

    ' Fungsi untuk cek apakah TextBox kosong
    Public Function IsTextBoxKosong(txt As TextBox) As Boolean
        If txt.Text.Trim() = "" Then
            Return True
        Else
            Return False
        End If
    End Function

    ' Fungsi untuk format rupiah
    Public Function FormatRupiah(angka As Double) As String
        Return "Rp " & angka.ToString("N0")
    End Function

End Module

```

Penggunaan di Form:

```

Private Sub btnSimpan_Click(...) Handles btnSimpan.Click
    ' Menggunakan fungsi dari Module
    If IsTextBoxKosong(txtNama) Then
        MessageBox.Show("Nama tidak boleh kosong!")
        Exit Sub
    End If

    Dim biaya As Double = 500000
    lblBiaya.Text = FormatRupiah(biaya)
End Sub

```

Nilai Efisiensi: Dengan Module, Anda menulis fungsi validasi **sekali**, tapi bisa digunakan di **10 Form berbeda**. Ini menghemat waktu development dan memastikan konsistensi validasi di seluruh aplikasi—bentuk **Amanah** dalam efisiensi kerja.

6. Implementasi Kode: Kasus Aplikasi Sederhana

Kasus 1 (Sub): Prosedur BersihkanForm()

Masalah: Anda memiliki Form dengan 10 TextBox. Setiap kali tombol Reset diklik, Anda harus menulis `TextBox1.Clear()`, `TextBox2.Clear()`, dst. Ini melanggar prinsip DRY!

Solusi: Buat satu prosedur yang mengosongkan semua TextBox.

Langkah 1: Buat Prosedur di Form

```
Sub BersihkanForm()  
    txtNIS.Text = ""  
    txtNama.Text = ""  
    txtKelas.Text = ""  
    txtAlamat.Text = ""  
  
    ' Set fokus ke TextBox pertama  
    txtNIS.Focus()  
End Sub
```

Langkah 2: Panggil dari Tombol Reset

```
Private Sub btnReset_Click(...) Handles btnReset.Click  
    BersihkanForm()  
End Sub
```

Langkah 3: Panggil Juga Setelah Simpan Data

```
Private Sub btnSimpan_Click(...) Handles btnSimpan.Click  
    ' ... kode simpan data ...  
    MessageBox.Show("Data berhasil disimpan!")  
    BersihkanForm() ' Siap untuk input data baru  
End Sub
```

Manfaat:

- Jika ada penambahan TextBox baru, cukup update 1 prosedur
- Kode tombol Reset lebih pendek dan mudah dibaca
- Konsistensi: Semua tombol yang butuh reset menggunakan prosedur yang sama

Kasus 2 (Function): Fungsi HitungStatusLulus(nilai)

Masalah: Anda perlu menampilkan status kelulusan di 5 tempat berbeda (Label, MessageBox, Laporan, dll). Menulis logika If-Else di setiap tempat adalah pemborosan.

Solusi: Buat Function yang mengembalikan status sebagai String.

Langkah 1: Buat Function

```
Function HitungStatusLulus(nilai As Integer) As String
    Const KKM As Integer = 75

    If nilai >= KKM Then
        Return "LULUS"
    Else
        Return "REMIDIAL"
    End If
End Function
```

Langkah 2: Gunakan di Berbagai Tempat

```
' Di Label
lblStatus.Text = HitungStatusLulus(CInt(txtNilai.Text))

' Di MessageBox
MessageBox.Show("Status Anda: " & HitungStatusLulus(nilai))

' Di ListBox
lstHasil.Items.Add(nama & " - " & HitungStatusLulus(nilai))

' Di variabel untuk laporan
Dim statusLaporan As String = HitungStatusLulus(nilai)
```

Manfaat:

- Jika KKM berubah, cukup ubah 1 tempat (di Function)
- Konsistensi: Semua tempat menampilkan status dengan logika yang sama
- Testing lebih mudah: Cukup test 1 Function, tidak perlu test 5 tempat

7. Penerapan Clean Code

7.1. Tips Menamai Prosedur dan Fungsi

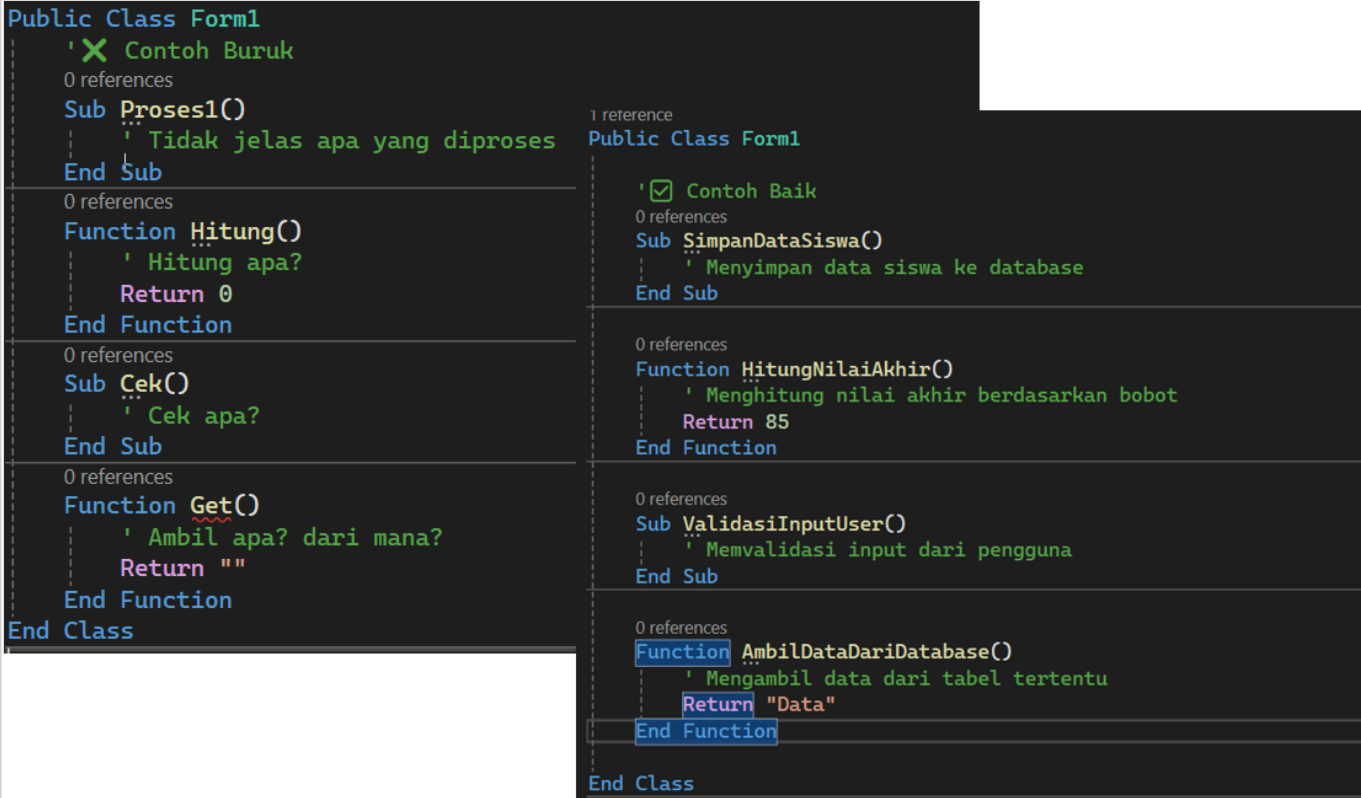
Nama yang baik menjelaskan **apa yang dilakukan**, bukan **bagaimana caranya**.

Nama Buruk	Nama Baik	Alasan
Sub Proses1()	Sub SimpanDataSiswa()	Jelas fungsinya

Nama Buruk	Nama Baik	Alasan
Function Hitung()	Function HitungNilaiAkhir()	Spesifik apa yang dihitung
Sub Cek()	Sub ValidasiInputUser()	Jelas apa yang divalidasi
Function Get()	Function AmbilDataDariDatabase()	Jelas sumber data

Konvensi Penamaan yang Disarankan:

Jenis	Format	Contoh
Sub Procedure	Kata Kerja + Objek	SimpanData(), HapusRecord()
Function	Kata Kerja + Objek + (opsional) Tipe	HitungTotal(), GetNamaSiswa()
Module	mod + Deskripsi	modDatabase, modValidasi



[GAMBAR 4.10: Perbandingan Nama Prosedur Baik vs Buruk]

7.2. Prinsip Clean Code dalam Modularisasi

Prinsip	Implementasi
Satu Fungsi, Satu Tugas	Jangan buat function yang sekaligus hitung, simpan, dan cetak
Maksimal 20 Baris	Jika fungsi terlalu panjang, pecah menjadi sub-fungsi
Hindari Magic Number	Gunakan konstanta untuk nilai tetap (KKM, pajak, dll)
Komentar yang Bermakna	Jelaskan "mengapa", bukan "apa" (kode sudah menjelaskan apa)
Konsistensi Indentasi	Gunakan 4 spasi untuk setiap level indentasi

Contoh Clean Code:

```
' ✗ BURUK: Terlalu panjang, banyak tugas
Sub ProsesData()
  ' Validasi
  If txtNama.Text = "" Then ...
  ' Hitung
  Dim total = ...
  ' Simpan
  ' Cetak
  ' Kirim email
End Sub

' ✓ BAIK: Terpisah per fungsi
Sub ProsesData()
  If Not ValidasiInput() Then Exit Sub
  Dim total = HitungTotal()
  SimpanKeDatabase(total)
  CetakLaporan(total)
End Sub
```

Nilai Profesionalisme: Kode yang bersih adalah cerminan **karakter profesional** Anda. Sebagai calon guru, Anda mengajarkan siswa untuk rapi dan sistematis. Mulai dari kode Anda sendiri!

8. Aktivitas Praktikum Mandiri

Tugas: Module Konversi Suhu untuk Laboratorium Digital

Deskripsi: Buatlah sebuah Module yang berisi fungsi-fungsi konversi suhu untuk aplikasi "Laboratorium Digital" yang membantu siswa belajar Fisika.

Spesifikasi Module:

Nama Fungsi	Parameter	Return	Deskripsi
CelsiusToFahrenheit	celsius As Double	Double	$C \rightarrow F: (C \times 9/5) + 32$
CelsiusToKelvin	celsius As Double	Double	$C \rightarrow K: C + 273.15$
FahrenheitToCelsius	fahrenheit As Double	Double	$F \rightarrow C: (F - 32) \times 5/9$
FormatSuhu	nilai As Double, satuan As String	String	Format output: "25.50 °C"

Langkah Pengerjaan:

- 1. Buat proyek baru bernama `LaboratoriumDigital`.
- 2. Tambahkan Module baru: `modKonversiSuhu.vb`.
- 3. Implementasikan keempat fungsi di atas.
- 4. Desain Form dengan komponen berikut:

Komponen	Nama (Name)	Text/Properties
Form	Form1	Text = "Konversi Suhu"
GroupBox	grpInput	Text = "Input Suhu"
TextBox	txtNilai	-
ComboBox	cboDari	Items: Celsius, Fahrenheit
ComboBox	cboKe	Items: Celsius, Fahrenheit, Kelvin
Button	btnKonversi	"Konversi"
Button	btnReset	"Reset"
Label	lblHasil	"Hasil: -"

5. Tulis kode untuk tombol Konversi yang memanggil fungsi dari Module.

Contoh Kode yang Diharapkan:

```
' Di Module modKonversiSuhu.vb
Module modKonversiSuhu

    Public Function CelsiusToFahrenheit(celsius As Double) As Double
        Return (celsius * 9 / 5) + 32
    End Function

    Public Function CelsiusToKelvin(celsius As Double) As Double
        Return celsius + 273.15
    End Function

    Public Function FahrenheitToCelsius(fahrenheit As Double) As Double
        Return (fahrenheit - 32) * 5 / 9
    End Function

    Public Function FormatSuhu(nilai As Double, satuan As String) As String
        Return nilai.ToString("F2") & " °" & satuan
    End Function

End Module
```

```
' Di Form1.vb
Private Sub btnKonversi_Click(...) Handles btnKonversi.Click
    Dim nilaiInput As Double = CDb1(txtNilai.Text)
    Dim dariSatuan As String = cboDari.SelectedItem.ToString()
    Dim keSatuan As String = cboKe.SelectedItem.ToString()
    Dim hasil As Double

    If dariSatuan = "Celsius" And keSatuan = "Fahrenheit" Then
        hasil = CelsiusToFahrenheit(nilaiInput)
```

```

ElseIf dariSatuan = "Celsius" And keSatuan = "Kelvin" Then
    hasil = CelsiusToKelvin(nilaiInput)
ElseIf dariSatuan = "Fahrenheit" And keSatuan = "Celsius" Then
    hasil = FahrenheitToCelsius(nilaiInput)
Else
    MessageBox.Show("Konversi tidak tersedia!")
Exit Sub
End If

lblHasil.Text = "Hasil: " & FormatSuhu(hasil, keSatuan.Substring(0, 1))
End Sub

Private Sub btnReset_Click(...) Handles btnReset.Click
    txtNilai.Clear()
    lblHasil.Text = "Hasil: -"
    txtNilai.Focus()
End Sub

```

Nilai Karakter: Aplikasi ini membantu siswa memahami konsep Fisika dengan lebih interaktif. Dengan membuat fungsi yang reusable, Anda menunjukkan sikap **pedagogis**—menciptakan alat yang dapat digunakan berulang kali untuk membantu banyak siswa.

9. Evaluasi & Rangkuman

A. Soal Analisis

Soal 1: Kapan Anda harus memilih **Function** daripada **Sub Procedure**? Berikan 3 contoh situasi nyata dalam aplikasi pendidikan!

Soal 2: Jelaskan perbedaan efek menggunakan **ByVal** dan **ByRef** pada parameter Function! Berikan contoh kapan ByRef diperlukan!

Soal 3: Apa keuntungan utama menggunakan **Module** untuk menyimpan fungsi global? Jelaskan dari segi maintenance dan konsistensi!

B. Soal Kasus: Sederhanakan Kode Berulang

Kasus: Perhatikan kode berikut yang ada di 5 tombol berbeda:

```

' Di btnSimpan_Click
If txtNama.Text = "" Then
    MessageBox.Show("Nama kosong!")
Exit Sub
End If

' Di btnEdit_Click
If txtNama.Text = "" Then
    MessageBox.Show("Nama kosong!")

```



```

        Exit Sub
    End If

    ' Di btnHapus_Click
    If txtNama.Text = "" Then
        MessageBox.Show("Nama kosong!")
        Exit Sub
    End If

    ' ... dan seterusnya di 2 tombol lain

```

Tugas: Sederhanakan kode di atas menjadi satu prosedur tunggal di dalam Module yang dapat dipanggil dari semua tombol!

C. Rangkuman Bab 4

No	Poin Kunci	Deskripsi Singkat
1	DRY Principle	Don't Repeat Yourself - tulis kode sekali, pakai berkali-kali.
2	Sub Procedure	Melakukan aksi, tidak mengembalikan nilai.
3	Function	Melakukan kalkulasi, mengembalikan nilai dengan Return .
4	Parameter	Variabel dalam deklarasi prosedur/fungsi.
5	Argumen	Nilai aktual yang dikirim saat memanggil.
6	ByVal	Mengirim salinan (default, lebih aman).
7	ByRef	Mengirim referensi (mengubah nilai asli).
8	Module	Wadah fungsi global yang bisa diakses semua Form.
9	Clean Code	Nama deskriptif, satu fungsi satu tugas, maksimal 20 baris.
10	Maintenance	Modularisasi memudahkan perubahan dan debugging.

Penutup

Selamat! Anda telah menyelesaikan Bab 4 tentang Pemrograman Modular. Sekarang Anda memiliki keterampilan untuk menulis kode yang **bersih, efisien, dan mudah dipelihara**. Ini adalah lompatan dari "programmer pemula" menuju "programmer profesional".

Prinsip modularisasi yang telah Anda pelajari akan sangat berguna ketika kita memasuki bab-bab berikutnya tentang **Basis Data MySQL**. Anda akan menggunakan Function dan Module untuk mengelola koneksi database, query, dan manipulasi data dengan cara yang terorganisir.

"Kode yang baik ditulis untuk manusia, bukan hanya untuk mesin." — Robert C. Martin

Teruslah berlatih menerapkan prinsip DRY dan Clean Code dalam setiap aplikasi yang Anda buat.